TRƯỜNG ĐẠI HỌC CÔNG NGHỆ
KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO MÔN HỌC

MÔN HỌC: KIỂM THỬ VÀ ĐẢM BẢO CHẤT LƯỢNG PHẦN MỀM

CÔNG CỤ KIỂM THỬ CPPCHECK

Giảng viên: TS. Nguyễn Thu Trang

Sinh viên: Mẫn Thị Bích Phương - 22021119 - K67I-IT2
Nguyễn Minh Hiển - 22021106 - K67I-IT15
Trần Anh Đức - 22021165 - K67I-IT2
Trần Thanh Thảo - 22021157 - K67I-IT2

Lớp học phần: INT3117 1

HÀ NỘI - 2025

Mục lục

1 Giới thiệu chung **2**

2 Kiến trúc hoạt động của CppCheck **3**

2.1 Tổng quan CppCheck 3

2.2 Tiền xử lý (Preprocessing) 4

2.3 Danh sách token (Token list) 5

2.4 Cây cú pháp (Syntax tree) 6

2.5 Cơ sở dữ liệu ký hiệu (Symbol Database) 7

2.6 Phân tích luồng giá trị (Value Flow) 8

2.7 Thư viện (Library) 9

3 Quy trình thực thi Cppcheck từ command line **10**

4 Một vài lỗi (Bugs) mà Cppcheck có thể phát hiện **12**

4.1 Phân tích dòng điều khiển (Control Flow Analysis) 12

4.2 Phát hiện lỗi tràn bộ đệm (Buffer Overflow) 12

4.3 Phân tích lời gọi hàm 13

4.4 Phát hiện rò rỉ bộ nhớ (Memory Leak) 13

4.5 Giới hạn của CppCheck 15

5 Đánh giá tổng quan **15**

5.1 Ưu điểm và Hạn chế 15

5.2 So sánh với các công cụ khác 16

5.3 Đối tượng phù hợp 17

6 Demo **17**

6.1 PVZ KaitoNMH Edition 18

6.2 SQLite3 19

6.3 Juliet Test Suite 20

7 Kết luận **22**

Tài liệu tham khảo **23**

Bảng 1: Phân công công việc nhóm

Thành viên	**Đóng góp**	**Phần trăm**
Mẫn Thị Bích Phương	• **Nhóm trưởng** • Viết báo cáo chính; tổng hợp và chỉnh sửa toàn bộ nội dung • Thuyết trình	25%
Nguyễn Minh Hiển	• Tìm hiểu, viết và chỉnh sửa nội dung phần 2, 3 • Phụ trách demo	25%
Trần Thanh Thảo	• Hoàn thiện nội dung phần 1, 2 • Thuyết trình	25%
Trần Anh Đức	• Thiết kế slide • Hoàn thiện nội dung phần 3 • Thuyết trình	25%

1 Giới thiệu chung

Cppcheck là một công cụ kiểm thử tự động dành cho mã nguồn viết bằng các ngôn ngữ C và C++, và mục tiêu chính của Cppcheck là phát hiện các lỗi thực sự trong mã nguồn và giảm thiểu các cảnh báo sai (false positive). Cppcheck nổi bật với khả năng phân tích các đoạn mã có cú pháp không theo tiêu chuẩn, bao gồm cả những đoạn code xen kẽ hợp ngữ (thường được dùng trong các dự án nhúng). Nhờ đó, công cụ này đặc biệt hữu ích trong các môi trường phát triển hệ thống nhúng có yêu cầu bắt buộc đối với sự ổn định và an toàn của mã nguồn.

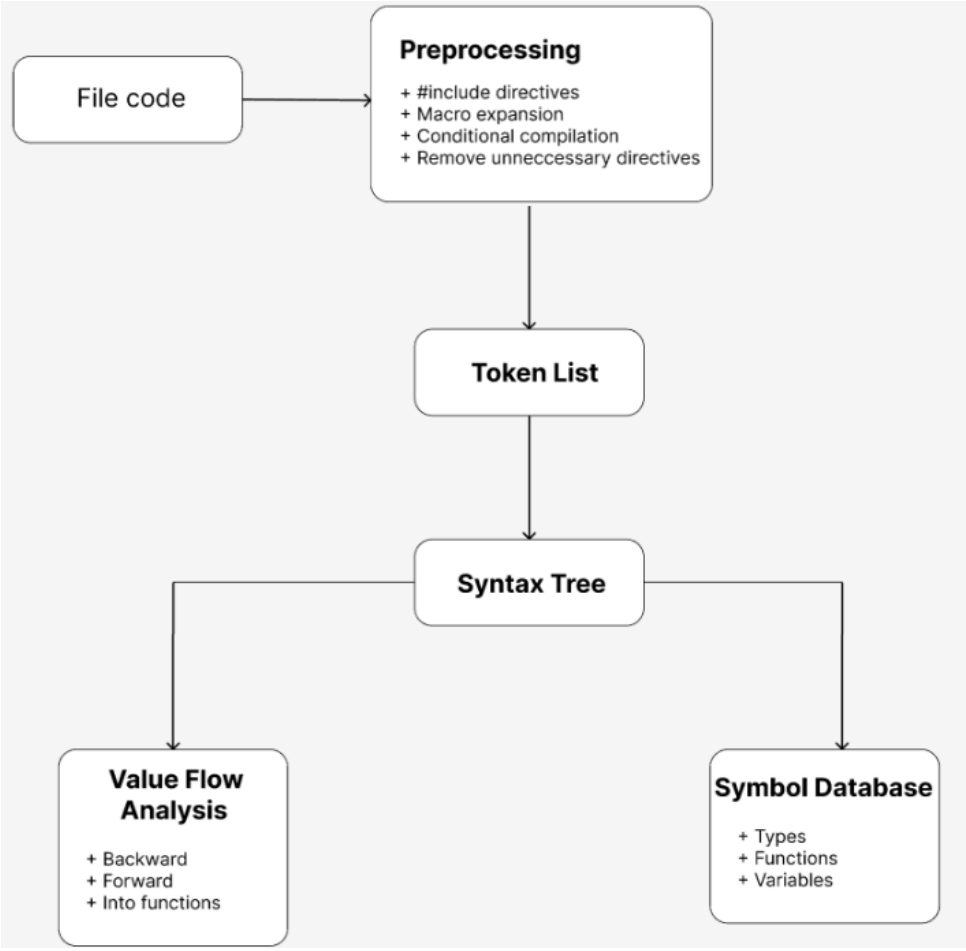
Cppcheck hoạt động dựa trên kỹ thuật phân tích tĩnh (static analysis) - tức là phân tích trực tiếp trên mã nguồn mà không tiến hành thực thi chương trình. Nhờ vào kỹ thuật này, công cụ có thể phát hiện các lỗi tiềm ẩn trong mã nguồn như là: hành vi không xác định, cấu trúc mã nguy hiểm, phong cách lập trình không phù hợp... ví dụ như truy cập bộ nhớ không hợp lệ hoặc sử dụng biến chưa khởi tạo, hoặc cppcheck sẽ đưa ra cảnh báo đối với những đoạn mã vi phạm quy chuẩn lập trình.

Tuy nhiên, không phải tất cả các lỗi đều có thể được phát hiện thông qua phân tích tĩnh. Các công cụ như Cppcheck không có khả năng hiểu được mục đích hay logic sâu xa của chương trình như con người. Vì vậy, nếu chương trình thực thi đúng cú pháp nhưng lại cho kết quả sai về mặt logic hoặc ý nghĩa (chẳng hạn in ra “Helo” thay vì “Hello”), thì các công cụ phân tích tĩnh thường không phát hiện được. Những lỗi dạng này thường thuộc về ngữ cảnh, lỗi đánh máy, hoặc sai sót logic mà không vi phạm cú pháp hay gây ra hành vi nguy hiểm rõ ràng.

Cppcheck hỗ trợ các chuẩn C++ hiện đại, từ C++11 trở lên, và có thể chạy tốt trên nhiều hệ điều hành như: Windows, Linux, macOS, và các hệ thống tuân thủ chuẩn POSIX. Ngoài ra, có thể tích hợp vào các công cụ CI/CD hoặc sử dụng trực tiếp trong môi trường phát triển như Visual Studio Code, Eclipse, hoặc thông qua dòng lệnh.

2 Kiến trúc hoạt động của CppCheck

2.1 Tổng quan CppCheck



Hình trên thể hiện sơ đồ hoạt động của Cppcheck với đầu vào là file chứa mã nguồn cần kiểm tra. File mã nguồn sẽ được tiền xử lý trước để loại bỏ các chỉ thị như *#include, #define*, mở rộng macro hay chèn nội dung của các file header vào mã nguồn. Sau đó, công cụ sẽ tạo một danh sách token từ mã đã tiền xử lý. Từ danh sách token này, cây cú pháp trừu tượng (Abstract Syntax Tree - AST) để biểu diễn cấu trúc logic của mã nguồn, được sử dụng để kiểm tra cú pháp và phân tích ngữ nghĩa rõ ràng và chính xác hơn. Ngoài ra, Cppcheck còn xây dựng một Symbol database lưu trữ các thông tin liên quan kiểu dữ liệu, hàm và biến trong chương trình.. Bên cạnh đó, Cppcheck thực hiện Value Flow Analysis để theo dõi sự thay đổi giá trị của biến trong suốt chương trình.

Các Checker sử dụng các thông tin có được từ các bước trên để tiến hành phân

tích và kiểm tra các lỗi có thể có tại mã nguồn và thông báo tới người dùng. Cuối cùng, Cppcheck cũng hỗ trợ người dùng định nghĩa các kiểm tra tùy chỉnh thông qua Library, lưu trữ các cấu hình bổ sung cho các hàm đặc biệt. Điều này cho phép công cụ mở rộng khả năng phát hiện lỗi với những hàm hoặc cấu trúc do người dùng định nghĩa một cách hiệu quả và chính xác hơn.

2.2 Tiền xử lý (Preprocessing)

Tại bước này, Cppcheck sử dụng **simplecpp**, một thư viện preprocessor mã nguồn mở của danmar, được bundling trong thư mục **externals/simplecpp** của dự án, với một số bước tiền xử lý chính sau:

- Xử lý chỉ thị *#include*: Với mỗi dòng *#include "file.h"* hoặc *#include <file.h>*, **simplecpp** sẽ tìm và chèn nội dung tệp header tương ứng từ các thư mục include được cung cấp. Nếu không tìm thấy tệp header, Cppcheck sẽ hiển thị cảnh báo nhưng không dừng quá trình tiền xử lý, cho phép tiếp tục phân tích phần còn lại của mã. Việc này khá hữu ích khi phân tích một dự án lớn, không phụ thuộc vào việc phải biên dịch đầy đủ các thư viện chuẩn mà dự án sử dụng, tập trung hơn vào mã nguồn chính của dự án.

- Mở rộng và thay thế macro (*#define, #undef*): Khi gặp các dòng lệnh #define, **simplecpp** sẽ lưu định nghĩa macro dưới dạng ánh xạ, ví dụ:

```
#define MAX 100   Nghĩa là cứ MAX thì sẽ ánh xạ đến 100
int a = MAX;   // Lệnh này sẽ được mở rộng thành: int a = 100;

```

  Mỗi khi token trùng tên macro xuất hiện, nó sẽ được mở rộng (expand) theo định nghĩa đã lưu. Tuy nhiên, do được thiết kế đơn giản, **simplecpp** không hỗ trợ toàn bộ tính năng nâng cao của preprocessor, ví dụ như COUNTER trong GCC.

- Xử lý điều kiện biên dịch (*#if, #ifdef, #elif, #endif*): **simplecpp** bao gồm trình phân tích và đánh giá biểu thức preprocessor, giúp xác định nên giữ hay loại bỏ các khối mã phụ thuộc điều kiện. Ví dụ:

```
\#ifdef DEBUG
printf("Debug mode\n");
\#endif

```

Nếu không có tùy chọn -DDEBUG, đoạn mã trên sẽ bị loại bỏ. Ngoài ra, Cppcheck cũng hỗ trợ người dùng cung cấp thêm các định nghĩa macro tùy chọn qua -D, -U hoặc –force để kiểm thử nhiều cấu hình biên dịch khác nhau.

- Loại bỏ các chỉ thị không cần thiết: Sau các bước xử lý trên, các chỉ thị như *#pragma, #line, #error*... sẽ bị loại bỏ hoàn toàn. Kết quả đầu ra là một dòng mã "phẳng", chỉ bao gồm các khai báo và logic thực thi, sẵn sàng cho các bước tiếp theo.

2.3 Danh sách token (Token list)

Sau khi mã nguồn được tiền xử lý, Cppcheck sẽ chuyển đổi mã nguồn thành một danh sách các token. Các token này bao gồm các thành phần cơ bản như từ khóa, tên biến, toán tử, dấu câu, và các thành phần khác của ngôn ngữ lập trình. Mỗi token đại diện cho một đơn vị ngữ nghĩa trong mã nguồn, giúp trình phân tích cú pháp có thể hiểu và kiểm tra mã một cách chính xác. Ví dụ, nếu mã nguồn là: abc = ab + c;. Cppcheck sẽ chuyển mã thành danh sách các token sau:

- abc
- =
- ab
- +
- c
- ;

Các checker trong Cppcheck sẽ duyệt qua các token này và có thể kiểm tra ngay một số vấn đề tiềm ẩn cơ bản, tiêu biểu như:

- Kiểm tra lỗi cú pháp (Syntax Checking): lỗi này được kiểu tra trong module Checker syntaxErrorChecker, như các lỗi thiếu dấu chấm phẩy, biểu thức không hợp lệ.... Cppcheck sẽ đưa ra error hoặc cảnh báo tuỳ thuộc vào loại lỗi.

- Kiểm tra kiểu dữ liệu (Type Checking): được thực hiện tại module checker typeChecker kiểm tra sự tương thích giữa kiểu dữ liệu của các toán hạng và toán tử. Ví dụ nếu ab là kiểu string và c là kiểu int, phép cộng ab + c có thể không hợp lệ và sẽ bị cảnh báo. Trong trường hợp abc, ab, và c đều là int, checker này sẽ không phát hiện lỗi.

- Kiểm tra quy tắc đặt tên biến (Naming Style Checking): được kiểm tra tại module checker namingStyleChecker, đưa ra cảnh bảo nếu mã nguồn không tuân theo quy chuẩn đặt tên của dự án (ví dụ: camelCase, snakecase, PascalCase, ...). Ví dụ nếu biến abc không phù hợp với chuẩn được yêu cầu, checker sẽ cảnh báo về cách đặt tên không nhất quán.

Để thực hiện các kiểm tra trên, Cppcheck sử dụng vòng lặp duyệt qua toàn bộ danh sách token trong đơn vị dịch (translation unit) như sau:

```
// Lặp qua tất cả các token
for (const Token *tok = tokenizer->tokens(); tok; tok = tok->next()) {
    ...
}
```

Trong đó:

- *tokenizer->tokens()*: Trả về danh sách token từ quá trình tiền xử lý.

- *tok*: Biến con trỏ trỏ tới từng token trong danh sách.

- *tok->next()*: Chuyển đến token tiếp theo trong danh sách.

2.4 Cây cú pháp (Syntax tree)

Dựa trên danh sách các token thu được từ bước phân tích từ vựng, Cppcheck tiến hành xây dựng AST.

AST là một cấu trúc dạng cây biểu diễn cấu trúc cú pháp của mã nguồn theo cách mà máy có thể hiểu được. Thông qua AST, Cppcheck có thể nắm được mối quan hệ

giữa các thành phần trong mã, chẳng hạn như biểu thức gán, câu lệnh điều kiện, vòng lặp, và các khối lệnh lồng nhau. Mỗi câu lệnh trong chương trình sẽ tương ứng với một cây cú pháp con trong AST tổng thể.

Các token từ danh sách ban đầu sẽ trở thành các nút trong cây cú pháp. Để dễ dàng truy xuất và phân tích, mỗi token có thể đi kèm các hàm thành viên hỗ trợ duyệt cây, tiêu biểu như:

- astParent(): Trả về nút cha của token trong AST, cho phép di chuyển lên cấp cao hơn trong cấu trúc cây.
- astOperand1() và astOperand2(): Trả về các toán hạng bên trái và bên phải (nếu có), thường dùng trong các biểu thức như phép gán, phép so sánh, hay phép toán học.

Việc xây dựng và sử dụng AST giúp các checker phát hiện được nhiều lỗi logic và cú pháp phức tạp mà chỉ phân tích tuyến tính token không thể nhận biết được. Ví dụ: ta có đoạn code:

```
int x = 5;
if (x = 10) {

}
```

Dòng *if (x = 10)* dùng phép gán (=) thay vì phép so sánh bằng (==), dẫn đến lỗi logic vì luôn trả về true và làm cho khối if luôn được thực hiện. Đây không phải lỗi cú pháp nên bước token không phát hiện được. Tuy nhiên, AST sẽ biểu diễn biểu thức *x = 10* như một biểu thức gán trong điều kiện của câu lệnh if, và checker sẽ kiểm tra cấu trúc đó.

2.5 Cơ sở dữ liệu ký hiệu (Symbol Database)

Trong quá trình phân tích mã nguồn, Cppcheck xây dựng một cơ sở dữ liệu ký hiệu (symbol database) nhằm lưu trữ thông tin chi tiết về các thành phần quan trọng trong chương trình. Cơ sở dữ liệu này đóng vai trò như một kho lưu trữ trung tâm, giúp các bộ kiểm tra (checker) dễ dàng truy xuất và phân tích các thông tin cần thiết. Cụ thể, cơ sở dữ liệu ký hiệu bao gồm:

- Kiểu dữ liệu (types): Ghi nhận các kiểu dữ liệu đã khai báo và sử dụng trong chương trình, bao gồm cả kiểu cơ bản và kiểu do người dùng định nghĩa.
- Hàm (functions): Lưu trữ thông tin về các hàm như tên hàm, kiểu trả về, danh sách tham số, vị trí định nghĩa,...
- Biến (variables): Ghi nhận tất cả các biến được khai báo, kiểu của chúng, phạm vi hoạt động, trạng thái khởi tạo,...

Các checker trong Cppcheck sử dụng cơ sở dữ liệu này để truy xuất thông tin chi tiết về các thành phần trên, từ đó thực hiện các kiểm tra hoặc phân tích như: Kiểm tra xem biến có được sử dụng đúng kiểu hay không, kiểm tra việc gọi hàm có đúng số lượng và kiểu tham số hay không, v.v

2.6 Phân tích luồng giá trị (Value Flow)

Trước khi thực hiện các kiểm tra, Cppcheck sẽ tiến hành phân tích ngữ cảnh và theo dõi giá trị khả dĩ của biến tại các điểm khác nhau trong chương trình. Phân tích này cho phép hệ thống hiểu rõ hơn về các luồng giá trị có thể xuất hiện, giúp phát hiện các lỗi tiềm ẩn chính xác hơn.

- Ngược dòng (backward): Theo dõi giá trị của biến từ các lệnh phía trước.
- Xuôi dòng (forward): Theo dõi giá trị của biến trong các lệnh tiếp theo.
- Vào trong các hàm: Theo dõi giá trị của biến khi hàm được gọi và khi biến được truyền vào.

Các giá trị suy luận được sẽ được lưu trữ trong các token tương ứng. Điều này giúp các checker của Cppcheck hiểu được ngữ nghĩa của từng biểu thức hoặc toán tử tại thời điểm phân tích, từ đó đưa ra cảnh báo chính xác hơn.

Phân tích này cho phép các công cụ như Cppcheck xác định các giá trị có thể xảy ra tại mỗi điểm trong chương trình, từ đó giúp phát hiện các lỗi như: chia cho 0, giá trị không hợp lệ và nâng cao khả năng kiểm thử tự động hóa, nhất là với các biểu thức phức tạp Ví dụ:

```
void f(int x) {
    int a = 10 / x;
}
void otherfunction() {
    f(0); f(2);
}
```

Khi phân tích hàm f, giá trị các token sẽ là:

- a: undefined, 5
- /: undefined, 5
- 10: 10
- x: 0,2

Sau khi phân tích và suy luận được các giá trị có thể có tại từng token trong mã nguồn, các checker của Cppcheck có thể dễ dàng phát hiện các lỗi tiềm ẩn như chia cho 0. Ví dụ, đối với phép chia, việc kiểm tra được thực hiện như trong đoạn code ví dụ sau:

```
// Có phải toán tử chia không?
if (token->str() == "/") {
    // Lấy toán hạng phải (RHS) của toán tử chia
    const Token *rhs = token->astOperand2();
    // Kiểm tra nếu RHS có giá trị "0"
    const ValueFlow::Value *val0 = rhs->getValue(0);
    if (val0 != NULL) {
        ... có giá trị "0" trong RHS ...
    }
}
```

2.7 Thư viện (Library)

Thư viện lưu trữ các thông tin cấu hình do người dùng cung cấp, chẳng hạn như: Một hàm có trả giá trị hay không. Hàm có cấp phát bộ nhớ hay không. Các tham số hợp lệ hoặc không hợp lệ. Những thông tin này không nên được mã hóa cứng trong mã nguồn mà nên được định nghĩa thông qua thư viện để dễ dàng thay đổi và mở rộng.

3 Quy trình thực thi Cppcheck từ command line

Khi bạn thực thi cppcheck từ command line, quá trình diễn ra như sau:

1. **Bắt đầu**: Người dùng nhập và thực thi lệnh
   `cppcheck [cácthamsố] [tệphoặcthưmục]`
   trên command line.

2. **Khởi tạo và phân tích tham số**:
   - Hàm thực thi chính `CppCheckExecutor::check` được kích hoạt.
   - `CmdLineParser::parseFromArgs` được gọi để đọc và hiểu các tham số (`[cácthamsố]`) mà người dùng đã cung cấp.
   - Các thiết lập và cấu hình từ tham số này được lưu trữ trong một đối tượng `Settings`.

3. **Lập danh sách tệp**: Sử dụng đối tượng `Settings` và đường dẫn (`[tệphoặcthưmục]`), `FileLister` được dùng để quét và tạo ra một danh sách đầy đủ các tệp mã nguồn cần được kiểm tra.

4. **Chuẩn bị thực thi (có thể song song)**: `ThreadExecutor` được kích hoạt. Tùy thuộc vào cấu hình (ví dụ: tham số `-j`), nó có thể chuẩn bị để chạy kiểm tra trên nhiều tệp cùng lúc bằng cách tạo ra và quản lý nhiều luồng hoặc tiến trình CppCheck.

5. **Xử lý từng tệp** (lặp lại cho mỗi tệp trong danh sách):
   - Đối với mỗi tệp nguồn: một instance `CppCheck` gọi hàm `CppCheck::check(<tệp>)` để bắt đầu xử lý tệp đó.
   - **Bước 5.1: Tiền xử lý (Preprocessor)** đã trình bày tại Mục 2.2
   - **Bước 5.2: Token hóa (Tokenizer)** đã trình bày tại Mục 2.3
   - **Bước 5.3: Chạy Các Bộ Kiểm Tra (Checkers)**
     - Hàm `runChecks()` của tất cả các lớp kiểm tra (mỗi lớp phụ trách một nhóm lỗi cụ thể) được thực thi.

- Tại đây, các phân tích sâu hơn như xây dựng AST, CFG và phân tích luồng dữ liệu diễn ra để cung cấp ngữ cảnh cho các Checker.

Hoạt động của các Checker:

- Mỗi Checker được thiết kế để tìm kiếm một loại lỗi cụ thể (ví dụ: null pointer dereference, division by zero, out-of-bounds access, memory leak, ...).
- Checker duyệt qua AST/CFG, sử dụng thông tin ngữ cảnh để đưa ra quyết định.
- *Khi nào báo lỗi*: nếu có đường thực thi nguy hiểm (ví dụ: con trỏ có thể null mà không kiểm tra).
- *Khi nào không báo lỗi*: nếu điều kiện nguy hiểm không thể xảy ra hoặc không đủ thông tin.

Ví dụ: Division by zero **Quy tắc:** Nếu toán hạng bên phải của phép chia là giá trị 0 → báo lỗi chia cho 0. **Thuật toán:** Dùng phân tích luồng giá trị để suy luận giá trị của biến tại thời điểm thực hiện phép chia.

- Kiểm tra token xem có phải là '/' không:
  `if (token->str() == "/")`
- Lấy toán hạng bên phải:
  `const Token *rhs = token->astOperand2();`
- Phân tích luồng giá trị xem toán hạng có thể là 0 không:
  `const ValueFlow::Value *val0 = rhs->getValue(0);`
- Nếu `val0 != Null` → có giá trị 0 trong toán hạng → báo lỗi.

- **Bước 5.4: Ghi nhận lỗi (nếu có)** Nếu một Checker phát hiện vấn đề, nó sẽ sử dụng interface `ErrorLogger` để báo cáo chi tiết về lỗi (loại lỗi, vị trí trong tệp, thông điệp, ...).

4 Một vài lỗi (Bugs) mà Cppcheck có thể phát hiện

4.1 Phân tích dòng điều khiển (Control Flow Analysis)

Phân tích luồng điều khiển là quá trình xác định các đường đi thực thi có thể xảy ra trong chương trình.

Cppcheck có sử dụng một mức độ phân tích luồng điều khiển cơ bản để xác định các đường thực thi khả dĩ trong mã. Dù phân tích này khá đơn giản, nó đủ để phát hiện một số lỗi cơ bản.

4.2 Phát hiện lỗi tràn bộ đệm (Buffer Overflow)

Nếu một mảng bị truy cập vượt quá giới hạn (out of bounds) tại một nơi nào đó trong phạm vi (scope) của nó, thì Cppcheck sẽ tạo ra một thông báo lỗi. Ví dụ đoạn mã sau:

```
void f() {
    char a[10];
    if (x + y == 2) {
        a[20] = 0;  // Truy cập ngoài phạm vi
    }
}
```

Cppcheck sẽ báo lỗi:

```
Array 'a[10]' index 20 out of bounds
```

Cppcheck không kiểm tra liệu điều kiện x + y == 2 có thể xảy ra hay không. Nó giả định rằng mọi câu lệnh đều có thể được thực thi và cảnh báo ngay khi có khả năng lỗi, ngay cả khi mã logic không cho phép điều kiện đó xảy ra.

Nó không cố gắng xác định liệu chương trình có thực sự thực thi đến dòng lệnh a[20] = 0; hay không. Nó giả định rằng mọi dòng lệnh đều có thể được thực thi. Vì thế, Cppcheck vẫn phát hiện lỗi ngay cả khi điều kiện x + y == 2 là điều không bao giờ xảy ra trong thực tế. Tuy vậy, Cppcheck vẫn coi đây là cảnh báo đúng, vì dòng lệnh gây lỗi thực sự tồn tại trong mã nguồn.

4.3 Phân tích lời gọi hàm

Cppcheck cũng kiểm tra các lời gọi hàm. Trong trường hợp đó, có thể cần phân tích luồng điều khiển để tránh cảnh báo sai. Xem ví dụ dưới đây, về mặt logic là tương tự ví dụ trước:

```
void f1(char *s) {
    s[20] = 0;
}
void f2() {
    char a[10];
    if (x + y == 2) {
        f1(a);  // Báo lỗi: Truy cập ngoài phạm vi
    }
}
```

Cppcheck sẽ báo lỗi:

```
Array 'a[10]' index 20 out of bounds
```

Vì nếu chương trình thực sự gọi tới f1(a) thì sẽ xảy ra lỗi. Tuy nhiên, nếu điều kiện được di chuyển vào trong hàm f1, thì Cppcheck sẽ không báo lỗi, vì nó không thể chứng minh rằng điều kiện x + y == 2 có thể xảy ra tại thời điểm gọi hàm từ f2. Ví dụ:

```
void f1(char *s) {
    if (x + y == 2) {
        s[20] = 0;  // Không báo lỗi
    }
}
void f2() {
    char a[10];
    f1(a);
}
```

Trong trường hợp này, không có thông báo lỗi nào được tạo ra.

4.4 Phát hiện rò rỉ bộ nhớ (Memory Leak)

Cppcheck sử dụng một hình thức phân tích luồng điều khiển đơn giản để kiểm tra rò rỉ bộ nhớ. Phân tích này giả định rằng mọi điều kiện có thể là đúng hoặc sai, và rằng mọi câu lệnh đều có thể được thực thi.

```
void f() {
    char *a = malloc(10);
    if (x + y == 2) {
        return;  // Có thể rò rỉ nếu đi theo nhánh này
    }
    free(a);
}
```

Cppcheck sẽ phát hiện lỗi sau:

```
Memory leak: a
```

Giải thích:

- Cppcheck không cố gắng phân tích xem chương trình có thực sự đi vào nhánh return; hay không.
- Nó chỉ nhìn thấy rằng nếu dòng return; được thực thi mà chưa gọi free(a); thì sẽ có rò rỉ bộ nhớ, vì bộ nhớ được cấp phát bởi malloc(10) chưa được giải phóng.

Vì vậy, Cppcheck báo lỗi rò rỉ bộ nhớ là chính xác, dù trong thực tế điều kiện x + y == 2 có thể không bao giờ xảy ra. Tuy nhiên, phân tích của Cppcheck bị hạn chế. Với những trường hợp phức tạp hơn như đoạn mã dưới đây:

```
void f(int x) {
    char *a = 0;
    if (x == 10)
        a = static_cast<char*>(malloc(10));
    if (x == 20)
        free(a);
}
```

Cppcheck sẽ không phát hiện ra lỗi nào.

Giải thích:

- Ở đây, biến a chỉ được cấp phát bộ nhớ nếu x == 10, nhưng chỉ được giải phóng nếu x == 20.
- Về mặt logic, rõ ràng rằng nếu x == 10, thì x == 20 sẽ không bao giờ đúng trong cùng một lần gọi hàm. → Sẽ có rò rỉ bộ nhớ nếu x == 10.

- Tuy nhiên, do Cppcheck giả định mọi điều kiện có thể đúng hoặc sai độc lập, nên nó không phát hiện ra rằng có sự không tương thích logic giữa hai điều kiện.
- Vì vậy, trong trường hợp này, Cppcheck bỏ sót lỗi rò rỉ bộ nhớ.
- Các công cụ phân tích tĩnh khác (như Clang Static Analyzer hoặc Coverity) có thể phát hiện lỗi trong ví dụ trên vì chúng sử dụng phân tích luồng điều khiển nâng cao hơn.

4.5 Giới hạn của CppCheck

Hiểu biết ngữ cảnh: Cppcheck không hiểu được những kiến thức thực tế (ví dụ: một ngày có 24 giờ). Phân tích dòng điều khiển đơn giản: Nó giả định rằng mọi điều kiện đều có thể đúng hoặc sai, hạn chế khả năng phát hiện các lỗi phức tạp. Không toàn diện: Cppcheck chỉ phát hiện một số lỗi, không thể thay thế các công cụ phân tích tĩnh cao cấp hơn hoặc quy trình kiểm tra mã nguồn thủ công.

5 Đánh giá tổng quan

5.1 Ưu điểm và Hạn chế

Một trong những điểm mạnh nhất của Cppcheck là khả năng hoạt động độc lập với trình biên dịch, giúp nó dễ dàng tích hợp vào bất kỳ dự án nào mà không cần cấu hình phức tạp. Nó không yêu cầu mã nguồn phải biên dịch được, điều này đặc biệt hữu ích đối với các dự án lớn hoặc mã chưa hoàn chỉnh. Bên cạnh đó, Cppcheck hỗ trợ phân tích các lỗi phổ biến như sử dụng bộ nhớ chưa được cấp phát, tràn bộ đệm, sử dụng biến chưa khởi tạo và các lỗi logic. Những tính năng này giúp giảm thiểu các lỗi khó phát hiện trong runtime, từ đó cải thiện độ tin cậy của phần mềm.

Cppcheck cũng cung cấp tùy chọn kiểm tra nâng cao với các loại phân tích như hiệu năng (performance), bảo mật (security), và tính năng chưa sử dụng (unused code). Điều này cho phép lập trình viên dễ dàng tùy chỉnh kiểm tra để phù hợp với nhu cầu cụ thể của dự án. Ví dụ, flag –enable=all giúp kích hoạt toàn bộ các loại kiểm tra để phân tích chuyên sâu, trong khi các flag cụ thể hơn như –enable=performance

hoặc –enable=warning có thể được sử dụng để tập trung vào các khía cạnh quan trọng.

Mặc dù có nhiều điểm mạnh, Cppcheck vẫn tồn tại một số hạn chế nhất định. Phân tích luồng điều khiển của Cppcheck còn khá đơn giản và không đủ mạnh để xác định chính xác các điều kiện logic phức tạp, dẫn đến bỏ sót một số lỗi tiềm ẩn. Công cụ cũng không có khả năng hiểu biết ngữ nghĩa hoặc kiến thức nền tảng – chẳng hạn như không biết rằng một ngày có 24 giờ, nên có thể bỏ qua các lỗi logic liên quan.

Ngoài ra, Cppcheck không theo dõi tốt các luồng thực thi đồng thời (multithreading) và khó phát hiện các lỗi phức tạp như race condition hay deadlock. Bên cạnh đó, Cppcheck cũng không phân tích được các lỗi liên quan đến thư viện ngoài hoặc mã chưa có sẵn nếu thiếu định nghĩa. Đây là lĩnh vực mà các công cụ như Infer của Facebook hoặc Helgrind (trong Valgrind) có thể làm tốt hơn. Vì vậy, khi dự án yêu cầu kiểm tra chuyên sâu về các vấn đề đa luồng, Cppcheck nên được sử dụng cùng với các công cụ khác để đạt hiệu quả tốt nhất.

Một trong những hạn chế lớn nhất khác là khả năng báo cáo sai (false positive), tức là báo cáo lỗi không tồn tại. Điều này có thể làm tăng công việc cho lập trình viên, đặc biệt trong các dự án lớn. Tuy nhiên, người dùng có thể giảm thiểu vấn đề này bằng cách cấu hình Cppcheck để loại trừ các kiểm tra không cần thiết thông qua file cấu hình hoặc các flag dòng lệnh.

5.2 So sánh với các công cụ khác

So với các công cụ phân tích thương mại như PVS-Studio hoặc Coverity Scan, Cppcheck có một số hạn chế nhất định. Ví dụ, nó không có khả năng phân tích bảo mật chuyên sâu như PVS-Studio, và các báo cáo của Cppcheck cũng không trực quan hoặc phong phú như những công cụ có giao diện đồ họa. Tuy nhiên, Cppcheck vẫn tỏ ra vượt trội ở những khía cạnh khác như sự linh hoạt, khả năng tùy chỉnh, và không phụ thuộc vào trình biên dịch.

Một điểm yếu của Cppcheck là khả năng báo cáo chi tiết về bảo mật và hiệu năng chưa mạnh bằng các công cụ cao cấp hơn. Tuy nhiên, với các dự án nhỏ hoặc khi yêu cầu không quá phức tạp, Cppcheck hoàn toàn đủ khả năng đáp ứng. Đặc biệt, trong các dự án có quy mô vừa và nhỏ, việc sử dụng Cppcheck giúp tiết kiệm thời gian và

chi phí phát triển mà vẫn đảm bảo chất lượng mã.

5.3 Đối tượng phù hợp

Cppcheck là công cụ phù hợp cho một loạt các đối tượng, từ lập trình viên cá nhân đến các nhóm phát triển phần mềm lớn. Với các cá nhân hoặc nhóm nhỏ, Cppcheck mang lại một giải pháp miễn phí và dễ sử dụng để cải thiện chất lượng mã. Với các nhóm lớn hoặc dự án thương mại, nó có thể được sử dụng như một công cụ bổ trợ, kết hợp với các công cụ phân tích thương mại để cung cấp nhiều góc nhìn khác nhau về chất lượng mã.

6 Demo

Để thực hiện kiểm thử với công cụ CppCheck, nhóm tiến hành demo trên 3 dự án sau:

PVZ KaitoNMH Edition: Đây là dự án BTL môn Lập trình nâng cao với C++ và framework SDL2 của bạn Nguyễn Minh Hiển trong nhóm. Đây là một project có khoảng 10.000 LOC.

SQLite3: SQLite3 là thư viện cơ sở dữ liệu mã nguồn mở viết hoàn toàn bằng C với hơn 200.000 dòng. SQLite3 nằm trong top 10 CSDL (DB-Engine April 2025).

Bộ Juliet Test Suite: Juliet Test Suite là một trong những tiêu chuẩn "vàng" để đánh giá các công cụ phân tích tĩnh với 64.099 test case trải đều trên hơn 100 loại lỗ hổng (CWE), trong gần 100.000 file mã nguồn. Bộ test này tập trung nhiều vào những lỗi liên quan đến lỗ hổng bảo mật trong code. Tỷ lệ phân bổ các nhóm lỗi (quản lý bộ nhớ, quản lý tài nguyên, đầu vào không tin cậy, giá trị nguy hiểm . . .) Được phát triển bởi NIST (Viện Tiêu chuẩn và Công nghệ Quốc gia Hoa Kỳ)/SARD (The Software Assurance Reference Dataset), kèm ground truth rõ ràng nên có thể dễ dàng precision và recall.

Toàn bộ kết quả liên quan nằm trong link:

https://github.com/nmhienbn/TestingAndQA/tree/main/DemoCppCheck

với các file kết quả là `cppcheck-report`.

6.1 PVZ KaitoNMH Edition

Config1:

```
cppcheck -j32 --cppcheck-build-dir=tmp --std=c++17 \
         --showtime=summary --check-level=exhaustive -Isrc src \
         2> cppcheck-report-1.txt
```

- Thời gian chạy: 7 giây
- Tùy chọn nổi bật:

  -j32: Chạy 32 luồng song song

  –cppcheck-build-dir=tmp: Lưu lại file tạm, tăng tốc chạy lại

  –check-level=exhaustive: Kiểm tra logic sâu (ví dụ: if lồng nhau)

  -I`<path>`**:** Khai báo thư viện

- Kết quả: chỉ có 4 `error`.

```
src\elements\plants\sunflower\sunflower.cpp:99:28: error: Uninitialized variable: temp.x_location [uninitvar]
    suns.push_back(temp);
                           ^
src\elements\plants\sunflower\sunflower.cpp:99:28: error: Uninitialized struct member: temp.x_location [uninitStructMember]
    suns.push_back(temp);
                           ^
src\elements\sun\sun.cpp:21:20: error: Uninitialized variable: temp.x_location [uninitvar]
    suns.push_back(temp);
                       ^
src\elements\sun\sun.cpp:21:20: error: Uninitialized struct member: temp.x_location [uninitStructMember]
    suns.push_back(temp);
                       ^
```

Listing 1: Lỗi Uninitialized variable/struct member

Khi xem lại thì thấy rằng `temp.xlocation` thật sự chưa được init khi khai báo. Thực tế là do C++ tự động init bằng 0.

Config2:

```
cppcheck --enable=all -j32 --cppcheck-build-dir=tmp --std=c++17 \
         --showtime=summary --check-level=exhaustive -Isrc src \
         2> cppcheck-report-2.xml
```

- Thời gian chạy: 8 giây
- Bổ sung tùy chọn:

  –enable=all: Bật kiểm tra toàn diện (style, performance, unused code, missing includes, v.v.)

- Kết quả: đã có nhiều lỗi hơn, được thống kê dưới đây:

Bảng 2: Thống kê số lần xuất hiện các loại cảnh báo

Count	Name
53	missingIncludeSystem
30	uninitMemberVar
21	missingOverride
8	missingInclude
8	unusedFunction
6	passedByValue
6	constVariableReference
5	unusedVariable
5	constParameterReference
...	...

6.2 SQLite3

Config:

```
cppcheck -j32 --cppcheck-build-dir=tmp \
         --std=c++17 --showtime=summary \
         --check-level=exhaustive src --xml \
         2> cppcheck-report-1.xml
```

- Thời gian chạy: 5 phút
- Lưu ý: Chạy đa luồng nhưng là nhiều file cùng lúc, chứ không phải từng phần trong file, nên nếu có file code dài thì chạy sẽ rất lâu.
- Thống kê kết quả:

Bảng 3: Thống kê các cảnh báo khác (Config1 XML)

Count	Name
6	nullPointerOutOfMemory
3	autoVariables
3	syntaxError
2	memleakOnRealloc
2	objectIndex
1	ctnullpointer
1	uninitvar
1	preprocessorErrorDirective

6.3 Juliet Test Suite

```
Thí nghiệm:
  cppcheck --enable=all -j32 --cppcheck-build-dir=tmp \
           --inconclusive --xml --xml-version=2 testcases/ \
           2> cppcheck-report.xml

  python checkerrors.py --manifest manifest.xml \
                        --xml cppcheck-report.xml
```

- Thời gian chạy: 3 giờ
- Kết quả đo được:

Bảng 4: Hiệu năng CppCheck (đã bao gồm `-inconclusive`)

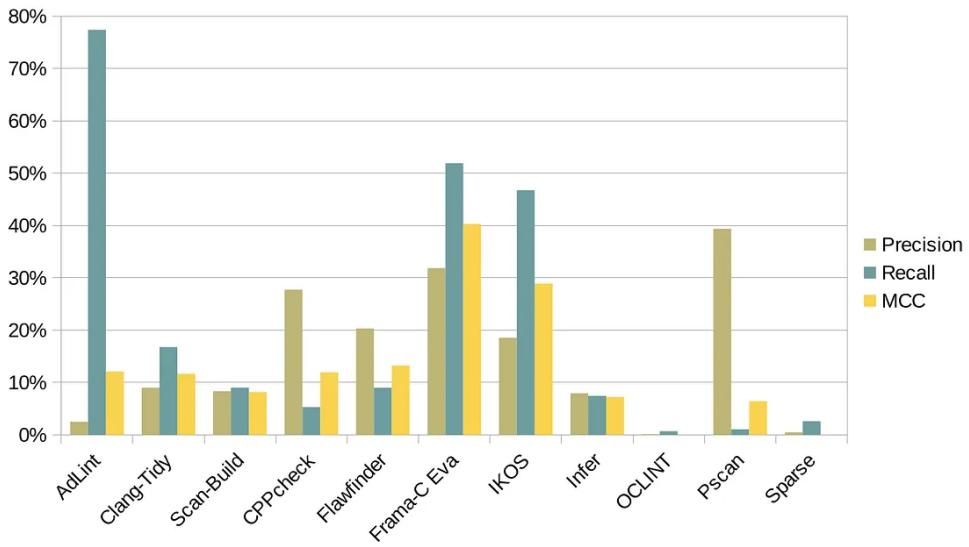
Metric	Value
TP	0.0138
FP	0.885
FN	0.101
Precision	0.02
Recall	0.12
F_1 Score	0.0343

- Nếu bỏ tùy chọn `-inconclusive` (tùy chọn bao gồm những kết quả mà cppcheck không chắc chắn) thì có thể thấy tỷ lệ FP đã giảm từ 0.885 xuống còn 0.248 và F1 Score cũng đã tăng lên.

Bảng 5: Hiệu năng CppCheck (không bao gồm `-inconclusive`)

Metric	Value
TP	0.058
FP	0.248
FN	0.694
Precision	0.19
Recall	0.08
F_1 Score	0.1126

- Kết quả trên cũng là kết quả chung cho những công cụ phân tích tĩnh cổ điển, chưa có sự ứng dụng của LLMs khi mang lên bàn cân so sánh với nhiều công cụ nổi tiếng khác trong một bộ test tập trung vào những lỗ hổng bảo mật như Juliet Test Suite.



Hình 1: Benchmark [1]

7 Kết luận

Cppcheck là một công cụ phân tích mã nguồn tĩnh mạnh mẽ và linh hoạt dành cho các dự án sử dụng ngôn ngữ C và C++. Đây là một giải pháp nổi bật trong danh mục các công cụ phân tích tĩnh nhờ khả năng tập trung vào việc phát hiện lỗi trong giai đoạn sớm của quá trình phát triển phần mềm, góp phần nâng cao chất lượng mã và giảm thiểu chi phí khắc phục lỗi. Dưới đây là tổng hợp chi tiết về các đặc điểm nổi bật của Cppcheck, từ ưu điểm đến các lĩnh vực ứng dụng, và cách nó phù hợp với từng nhóm người dùng.

Cppcheck hoạt động tốt trong cả môi trường phát triển cá nhân và nhóm, cũng như trong các dự án mã nguồn mở hoặc thương mại. Với kích thước nhỏ gọn và không yêu cầu cài đặt phức tạp, nó dễ dàng được tích hợp vào các quy trình CI/CD hiện đại như Jenkins, GitHub Actions, GitLab CI/CD. Điều này cho phép tự động kiểm tra mã nguồn mỗi khi có sự thay đổi, giúp phát hiện lỗi nhanh chóng trước khi mã được triển khai hoặc tích hợp.

Công cụ này đặc biệt hữu ích cho các nhóm nhỏ và các nhà phát triển độc lập, nơi chi phí cho các công cụ thương mại như PVS-Studio hoặc Coverity Scan có thể không khả thi. Đồng thời, tính năng miễn phí và mã nguồn mở của Cppcheck cũng làm cho nó trở thành một lựa chọn lý tưởng cho các dự án mã nguồn mở, nơi các nhóm phát triển cần một công cụ mạnh mẽ nhưng không yêu cầu ngân sách lớn.

Trong bối cảnh phát triển phần mềm hiện đại, nơi các tiêu chuẩn bảo mật và chất lượng ngày càng trở nên quan trọng, việc sử dụng Cppcheck như một phần của quy trình phát triển phần mềm không chỉ giúp phát hiện lỗi sớm mà còn giúp xây dựng niềm tin với khách hàng. Bằng cách tích hợp Cppcheck vào pipeline CI/CD, các nhóm phát triển có thể đảm bảo rằng mọi đoạn mã mới được thêm vào đều tuân thủ các tiêu chuẩn chất lượng đã định sẵn.

Cppcheck không phải là công cụ hoàn hảo, nhưng nó là một trong những công cụ phân tích mã nguồn tĩnh mạnh mẽ nhất trong tầm giá (miễn phí) hiện nay. Với khả năng phát hiện lỗi tốt, dễ tích hợp, và khả năng tùy chỉnh cao, Cppcheck là lựa chọn tuyệt vời cho các dự án muốn cải thiện chất lượng mã mà không cần đầu tư lớn. Tuy vẫn tồn tại một số hạn chế, nhưng với sự phát triển liên tục và cộng đồng hỗ trợ mạnh mẽ, Cppcheck chắc chắn sẽ tiếp tục là một công cụ quan trọng trong bộ công cụ của các nhà phát triển C/C++ hiện đại.

Tài liệu tham khảo

[1] Gentsch, C.; Krishnamurthy, R.; Heinze, T. S. (2021). Benchmarking Open-Source Static Analyzers for Security Testing for C. In: Margaria, T., Steffen, B. (eds) *Leveraging Applications of Formal Methods, Verification and Validation: Tools and Trends. ISoLA 2020.* Lecture Notes in Computer Science, vol. 12479. Springer, Cham. https://doi.org/10.1007/978-3-030-83723-5_13